PTHREADS

~~1 - Test practic APD~~

Să se implementeze, **folosind Pthreads**, un algoritm paralel pentru determinarea numărului de apariții ale unei valori într-un vector cu N elemente, precum și a sumei pozițiilor la care valoarea se găsește în vector, folosind P thread-uri. Rezolvarea se va face maximizând concurența thread-urilor. Rezultatul se va printa de către un thread ales de către programator (poate fi thread-ul cu un anumit ID), dar **NU** de către thread-ul *main*. Mai mult, doar thread-ului ales îi va fi permis să citească valorile parțiale calculate de celelalte thread-uri.

Atenție! Thread-urile **nu** au voie să incrementeze niciun contor partajat de către toate, deoarece incrementarea lui presupune citirea valorii inițiale.

În urma compilării, va rezulta un executabil numit *testapd*, care se va rula astfel: **`./testapd N P X`** (unde X este valoarea căutată).

Vectorul în care se caută valoarea va fi inițializat în funcția *main* ca **`v[i] = (i + 3) % 5`**, înainte de pornirea thread-urilor.

Programul va trebui să afișeze numărul de apariții ale elementului căutat în vector, apoi suma pozițiilor aparițiilor, cu spațiu între ele. Aveți mai jos trei exemple de rulare:

```
$ ./testapd 10 3 3
2 5

$ ./testapd 25 2 4
5 55

$ ./testapd 3 8 0
1 2
```

În primul exemplu de mai sus, vectorul este [3,4,0,1,2,3,4,0,1,2], deci elementul căutat (adică 3) se află la pozițiile 0 și 5. În acest caz, numărul de apariții este 2, iar suma acestora este 5.

Atenție! Toate thread-urile trebuie pornite în același timp (într-o buclă comună). Programul submis nu trebuie să printeze altceva decât numărul de apariții și suma pozițiilor aparițiilor, în această ordine și în formatul de mai sus. Afișarea se va face dintr-unul din thread-urile pornite de către voi.

Atenție! Formula cu care se inițializează vectorul este doar pentru a ușura testarea. Nu o folosiți pentru a deduce matematic numărul de apariții. Fiecare thread trebuie să itereze prin fragmentul de vector alocat lui și să găsească aparițiile valorii căutate.

Se presupune că parametrii de intrare sunt corecți (atât ca număr și ordine, cât și ca valori). Valorile acceptate pentru N, P și X sunt: $0 < N <= 10000$, $0 < P <= 8$, $0 <= X < 5$. Programul trebuie să funcționeze corect pentru orice combinație de N, P și X din aceste intervale.

Trebuie să încărcați pe Moodle o arhivă care să conțină **în rădăcină** fișierele sursă, un Makefile cu target-ul *build* și eventual un fișier README.

Atenție! Fișierul README este necesar doar în cazul în care considerați că este nevoie de explicații adiționale în legătură cu implementarea realizată.

~~2 - Test practic APD~~

Să se implementeze, **folosind Pthreads**, un program paralel cu P thread-uri, în care ultimele P-1 thread-uri realizează un job abstract (un *sleep* cu durata între 1 și 7 secunde, așa cum se va specifica mai jos), iar **thread-ul 0** afișează mesajul „Thread-ul X a terminat” **imediat** ce fiecare dintre ele a terminat execuția job-ului abstract.

În urma compilării, va rezulta un executabil numit *testapd*, care va primi ca parametri valoarea lui P și durata de *sleep* (în secunde) a fiecăruia din ultimele P-1 thread-uri. Aveți mai jos trei exemple de rulare:

```
$ ./testapd 3 2 4
Thread-ul 1 a terminat
Thread-ul 2 a terminat

$ ./testapd 3 4 2
Thread-ul 2 a terminat
Thread-ul 1 a terminat

$ ./testapd 8 7 6 5 4 3 2 1
Thread-ul 7 a terminat
Thread-ul 6 a terminat
Thread-ul 5 a terminat
Thread-ul 4 a terminat
Thread-ul 3 a terminat
Thread-ul 2 a terminat
Thread-ul 1 a terminat
```

În primul exemplu de rulare de mai sus, se specifică faptul că vor fi trei thread-uri. Dintre acestea, thread-ul 1 va executa un job abstract (adică va face un *sleep*) de 2 secunde, iar thread-ul 2 va avea alocat un job de 4 secunde. În momentul în care thread-ul 1 termină de executat job-ul (adică la 2 secunde după pornirea sa), thread-ul 0 afișează mesajul „Thread-ul 1 a terminat”. După încă aproximativ 2 secunde, thread-ul 2 termină și el execuția, moment în care thread-ul 0 afișează „Thread-ul 2 a terminat”.

Atenție! Toate thread-urile trebuie pornite în același timp (într-o buclă comună). Doar thread-ul 0 are voie sa printeze. Programul submis nu trebuie să printeze altceva decât mesajele de tipul „Thread-ul X a terminat” (câte unul pe linie).

Se presupune că parametrii de intrare sunt corecți (atât ca număr și ordine, cât și ca valori). Valorile acceptate pentru P sunt 2 <= P <= 8, iar pentru durata unui job sunt între 1 și 7 secunde (inclusiv).

Atenție! Se va trata și situația în care thread-ul de afișare este lent, iar mai multe alte thread-uri termină simultan, ele trebuind să **nu** se blocheze în așteptarea lui. O soluție care respectă logica de paralelism, dar nu respectă această condiție, va fi depunctată cu 20% din punctaj.

Trebuie să încărcați pe Moodle o arhivă care să conțină **în rădăcină** fișierele sursă, un Makefile cu target-ul *build* și eventual un fișier README.

Atenție! Fișierul README este necesar doar în cazul în care considerați că este nevoie de explicații adiționale în legătura cu implementarea realizată.

~~3 - Test practic APD~~

Să se implementeze, **folosind Pthreads**, un program paralel pentru sortarea unui vector cu N elemente prin intermediul Rank Sort, folosind P thread-uri. Rezolvarea se va face maximizând concurența thread-urilor.

Rezultatul se va printa de către un thread ales de către programator (poate fi thread-ul cu un anumit ID), dar **NU** de către thread-ul *main*. Mai mult, doar acestui thread îi va fi permis să citească fragmentele parțiale de vector sortate de celelalte thread-uri.

În urma compilării, va rezulta un executabil numit *testapd*, care va primi ca parametri valorile lui **N** și **P** (în această ordine).

Vectorul de sortat va fi inițializat în funcția *main* ca `v[i] = N - i + 5`, înainte de pornirea thread-urilor.

Aveți mai jos trei exemple de rulare:

```
$ ./testapd 10 3
6 7 8 9 10 11 12 13 14 15

$ ./testapd 25 4
6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30

$ ./testapd 3 8
6 7 8
```

Atenție! Toate thread-urile trebuie pornite în același timp (într-o buclă comună). Programul submis nu trebuie să printeze altceva decât vectorul sortat. Afișarea vectorului sortat se va face dintr-unul din thread-urile pornite de către voi.

Se presupune că parametrii de intrare sunt corecți (atât ca număr și ordine, cât și ca valori). Valorile acceptate pentru N și P sunt: $0 < N <= 10000$, $0 < P <= 8$. Programul trebuie să funcționeze corect pentru orice combinație de N și P din aceste intervale și pentru orice vector care **nu** conține duplicate.

Trebuie să încărcați pe Moodle o arhivă care să conțină **în rădăcină** fișierele sursă, un Makefile cu target-ul *build* și eventual un fișier README.

Atenție! Fișierul README este necesar doar în cazul în care considerați că este nevoie de explicații adiționale în legătura cu implementarea realizată.

~~4 - Test practic APD~~

Să se implementeze, **folosind Pthreads**, un program paralel cu N thread-uri în care fiecare thread generează M perechi de numere naturale (K, V) din intervalele [1, 10], respectiv [1, 100000]. La final, programul va afișa **exact** 10 perechi de forma (I, V_max_i) unde V_max_i este maximul valorilor V generate în perechile (I, V) din toate thread-urile.

Valorile M se vor alege aleator între 2 și 10 de către fiecare thread, iar rezultatul trebuie să se printeze o singură dată, la final. Implementarea va maximiza concurența între thread-uri.

Introducerea unei valori în cadrul unui thread va respecta **obligatoriu** următorul pseudocod:

- generează aleator o pereche nouă (K, V)
- printează un mesaj de forma `[Thread %d] a generat perechea (%d, %d)`
- citește valoarea maximă anterioară pentru cheia K
- așteaptă (folosind `sleep`) o secundă
- actualizează valoarea nouă de la cheia K.

În urma compilării, va rezulta un executabil numit *testapd*, care se va rula astfel: **`./testapd N`** (unde N este numărul de thread-uri). Se presupune că programul este apelat corect și trebuie să funcționeze corect pentru orice valoare a lui N între 1 și 8 (inclusiv). Programul va trebui să afișeze mesajul de mai sus de fiecare dată când un thread generează o pereche nouă, precum și rezultatul final. Aveți mai jos un exemplu de rulare:

```
$ ./testapd 3
[Thread 0] a generat perechea (2, 7)
[Thread 1] a generat perechea (1, 10)
[Thread 2] a generat perechea (3, 9)
[Thread 0] a generat perechea (1, 5)
[Thread 1] a generat perechea (3, 11)
[Thread 2] a generat perechea (1, 11)
[Thread 0] a generat perechea (8, 25)
[Thread 1] a generat perechea (2, 10)

(1, 11), (2, 10), (3, 11), (4, 0), (5, 0), (6, 0), (7, 0), (8, 25), (9, 0), (10, 0)
```

În exemplul de mai sus, thread-urile 0 și 1 au ales aleator M=3, iar thread-ul 2 a ales M=2. Așa cum se observă, rezultatul final conține înșiruirea celor 10 chei și valorile maxime din toate thread-urile pentru fiecare din ele. Pentru cheile fără valori (4, 6, 7, 9 și 10 din exemplul de mai sus) se consideră valoarea implicită 0.

Atenție! Implementarea trebuie să maximizeze paralelismul. În exemplu s-au făcut 8 operații cu durata minimă de câte o secundă, însă datorită ordinii lor, toate thread-urile pot executa (chiar și cu respectarea pseudocodului de adăugare) mai multe operații la fiecare moment, deci timpul rezultat va fi (cel mai probabil) mai mic. O implementare care serializează operațiile nu va fi acceptată.

Trebuie să încărcați pe Moodle o arhivă care să conțină **în rădăcină** fișierele sursă, un Makefile cu target-ul *build* și eventual un fișier README. Fișierul README este necesar doar în cazul în care considerați că este nevoie de explicații adiționale în legătură cu implementarea realizată.

~~5 - Test practic APD~~

Să se implementeze, **folosind Pthreads**, un algoritm paralel pentru determinarea numărului de valori prime dintr-un vector cu N elemente, precum și a sumei pozițiilor la care numerele prime se găsesc în vector, folosind P thread-uri. Rezolvarea se va face maximizând concurența thread-urilor. Rezultatul se va printa de către un thread ales de către programator (poate fi thread-ul cu un anumit ID), dar **NU** de către thread-ul *main*. Mai mult, doar acestui thread îi va fi permis să citească valorile parțiale calculate de celelalte thread-uri.

Atenție! Thread-urile nu au voie să incrementeze niciun contor unic **partajat de către toate**, deoarece incrementarea lui presupune citirea valorii inițiale, citirea valorilor calculate de alte thread-uri fiind permisă doar thread-ului desemnat. Nerespectarea acestei cerințe va duce la un punctaj de **0** pe problemă.

Atenție! Thread-ul *main* nu are voie să afișeze nimic. Nerespectarea acestei cerințe va duce la un punctaj de **0** pe problemă.

În urma compilării, va rezulta un executabil numit *testapd*, care se va rula astfel: **`./testapd N P n1 n2 ... nN`** (n1…nN sunt cele N numere din vector). Programul va trebui să afișeze numărul de elemente prime din vector, apoi suma pozițiilor acestora, cu spațiu între ele. Aveți mai jos trei exemple de rulare:

```
$ ./testapd 10 3 2 6 3 13 21 11 14 1 17 8
5 18

$ ./testapd 3 4 13 14 15
1 0

$ ./testapd 5 2 29 32 11 4 18
2 2
```

În primul exemplu de mai sus, vectorul este [2,6,3,13,21,11,14,1,17,8], deci elementele prime se află la pozițiile 0, 2, 3, 5 și 8. În acest caz, numărul de elemente prime este 5, iar suma pozițiilor acestora este 18.

Atenție! Toate thread-urile trebuie pornite în același timp (într-o buclă comună). Programul submis nu trebuie să printeze altceva decât numărul de elemente prime și suma pozițiilor acestora, în această ordine și în formatul de mai sus. Afișarea se va face dintr-unul din thread-urile pornite de către voi.

Se presupune că parametrii de intrare sunt corecți (atât ca număr și ordine, cât și ca valori). Valorile acceptate pentru *N*, *P* și *ni* sunt: $0 < N <= 100$, $0 < P <= 8$, $0 <= ni < 100$. Programul trebuie să funcționeze corect pentru orice combinație de *N*, *P* și *ni* din aceste intervale.

Trebuie încărcată pe Moodle o arhivă care să conțină **în rădăcină** sursele, un Makefile cu target-ul *build* și eventual un fișier README.

Atenție! Fișierul README este necesar doar în cazul în care considerați că este nevoie de explicații adiționale în legătură cu implementarea realizată. Indiferent dacă aveți sau nu fișier README, este important să adăugați comentarii în cod.

~~6 - Test practic APD~~

Să se implementeze, **folosind Pthreads**, un program paralel pentru realizarea operației de Reduce (pentru sumă, produs, minim și maxim) asupra unui vector cu N elemente, folosind P thread-uri. Rezolvarea se va face maximizând concurența thread-urilor. Rezultatul se va printa de către un thread ales de către programator (poate fi thread-ul cu un anumit ID), dar **NU** de către thread-ul *main*. Mai mult, doar acestui thread îi va fi permis să citească valorile parțiale calculate de celelalte thread-uri.

Atenție! Thread-urile nu au voie să acumuleze într-un contor unic **partajat de către toate**, deoarece modificarea lui presupune citirea valorii inițiale, citirea valorilor calculate de alte thread-uri fiind permisă doar thread-ului desemnat. Nerespectarea acestei cerințe va duce la un punctaj de **0** pe problemă.

Atenție! Thread-ul *main* nu are voie să afișeze nimic. Nerespectarea acestei cerințe va duce la un punctaj de **0** pe problemă.

În urma compilării, va rezulta un executabil numit *testapd*, care se va rula astfel: **`./testapd N P O n1 n2 ... nN`** (O reprezintă operația aplicată asupra tuturor elementelor, iar n1…nN sunt cele N numere din vector). Programul va trebui să afișeze rezultatul aplicării operației O asupra tuturor elementelor din vector. Aveți mai jos trei exemple de rulare:

```
$ ./testapd 10 3 sum 2 6 3 13 21 11 14 1 17 8
96

$ ./testapd 3 4 prod 2 10 4
80

$ ./testapd 5 2 min 29 32 11 4 18
4
```

În primul exemplu de mai sus, vectorul este [2,6,3,13,21,11,14,1,17,8] și operația dorită este suma, deci rezultatul afișat va fi 96.

Atenție! Toate thread-urile trebuie pornite în același timp (într-o buclă comună). Programul submis nu trebuie să printeze altceva decât rezultatul operației cerute asupra tuturor elementelor din vector. Afișarea se va face dintr-unul din thread-urile pornite de către voi.

Se presupune că parametrii de intrare sunt corecți (atât ca număr și ordine, cât și ca valori). Valorile acceptate pentru *N*, *P* și *ni* sunt: 0 < *N* <= 100, 0 < *P* <= 8, 0 <= *ni* < 100. Operația poate fi *sum*, *prod*, *min*, sau *max*. Programul trebuie să funcționeze corect pentru orice combinație de *N*, *P* și *ni* din aceste intervale, pentru orice operație.

Trebuie încărcată pe Moodle o arhivă care să conțină **în rădăcină** sursele, un Makefile cu target-ul *build* și eventual un fișier README.

Atenție! Fișierul README este necesar doar în cazul în care considerați că este nevoie de explicații adiționale în legătură cu implementarea realizată. Indiferent dacă aveți sau nu fișier README, este important să adăugați comentarii în cod.

7 - Test practic APD

Să se implementeze, **folosind Pthreads**, un program paralel pentru realizarea operației de Scan (pentru sumă) asupra unui vector cu N elemente, folosind P thread-uri. Rezolvarea se va face maximizând concurența thread-urilor. Rezultatul se va printa de către un thread ales de către programator (poate fi thread-ul cu un anumit ID), dar **NU** de către thread-ul *main*.

Atenție! Thread-ul *main* nu are voie să afișeze nimic. Nerespectarea acestei cerințe va duce la un punctaj de **0** pe problemă.

În urma compilării, va rezulta un executabil numit *testapd*, care se va rula astfel: **`./testapd N P n1 n2 ... nN`** (n1…nN sunt cele N numere din vector). Programul va trebui să afișeze toate sumele parțiale, în ordine. Aveți mai jos trei exemple de rulare:

```
$ ./testapd 10 3 2 6 3 13 21 11 14 1 17 8
2 8 11 24 45 56 70 71 88 96

$ ./testapd 3 4 2 10 4
2 12 16

$ ./testapd 5 2 29 32 11 4 18
29 61 72 76 94
```

În primul exemplu de mai sus, vectorul este [2,6,3,13,21,11,14,1,17,8], deci se afișează toate sumele parțiale în ordine (primul element, suma primelor două elemente, suma primelor trei elemente, etc.).

Atenție! Toate thread-urile trebuie pornite în același timp (într-o buclă comună). Programul submis nu trebuie să printeze altceva decât rezultatele parțiale cerute. Afișarea se va face dintr-unul din thread-urile pornite de către voi.

Se presupune că parametrii de intrare sunt corecți (atât ca număr și ordine, cât și ca valori). Valorile acceptate pentru *N*, *P* și *ni* sunt: $0 < N <= 100$, $0 < P <= 8$, $0 <= ni < 100$. Programul trebuie să funcționeze corect pentru orice combinație de *N*, *P* și *ni* din aceste intervale.

Trebuie încărcată pe Moodle o arhivă care să conțină **în rădăcină** sursele, un Makefile cu target-ul *build* și eventual un fișier README.

Atenție! Fișierul README este necesar doar în cazul în care considerați că este nevoie de explicații adiționale în legătură cu implementarea realizată. Indiferent dacă aveți sau nu fișier README, este important să adăugați comentarii în cod.

~~8 - Test practic APD~~

Să se implementeze, **folosind Pthreads**, un program paralel care caută, într-un vector cu N elemente, numerele divizibile cu anumite valori, folosind P thread-uri. Rezolvarea se va face maximizând concurența thread-urilor. Rezultatul se va printa de către un thread ales de către programator (poate fi thread-ul cu un anumit ID), dar **NU** de către thread-ul *main*. Mai mult, doar acestui thread îi va fi permis să citească valorile calculate de celelalte thread-uri.

Atenție! Thread-ul *main* nu are voie să afișeze nimic. Nerespectarea acestei cerințe va duce la un punctaj de **0** pe problemă.

În urma compilării, va rezulta un executabil numit *testapd*, care se va rula astfel: **`./testapd N P n1 n2 ... nN`** (n1…nN sunt cele N numere din vector). Programul va trebui să afișeze toate numerele divizibile cu 2, …, P+1, iar un thread cu ID-ul ID va fi responsabil de verificarea divizibilității pentru divizorul ID+2. Aveți mai jos trei exemple de rulare:

```
$ ./testapd 10 3 2 6 3 13 21 11 14 1 17 8
2: 2 6 14 8
3: 6 3 21
4: 8

$ ./testapd 3 2 2 10 4
2: 2 10 4
3:

$ ./testapd 5 4 25 32 11 4 18
2: 32 4 18
3: 18
4: 32 4
5: 25
```

În primul exemplu de mai sus, vectorul este [2,6,3,13,21,11,14,1,17,8] și se rulează cu 3 thread-uri, deci se vor căuta numerele divizibile cu 2, 3 și 4, care se afișează în formatul prezentat.

Atenție! Toate thread-urile trebuie pornite în același timp (într-o buclă comună). Programul submis nu trebuie să printeze altceva decât rezultatele cerute. Afișarea se va face dintr-unul din thread-urile pornite de către voi.

Atenție! Doar thread-ul cu ID-ul ID trebuie să verifice ce numere sunt divizibile cu ID+2. Astfel, în primul exemplu de mai sus, thread-ul 0 verifică numerele divizibile cu 2, thread-ul 1 pe cele divizibile cu 3, și thread-ul 2 pe cele divizibile cu 4, iar unul singur din cele trei thread-uri face afișarea.

Se presupune că parametrii de intrare sunt corecți (atât ca număr și ordine, cât și ca valori). Valorile acceptate pentru *N*, *P* și *ni* sunt: $0 < N <= 100$, $0 < P <= 8$, $0 <= ni < 100$. Programul trebuie să funcționeze corect pentru orice combinație de *N*, *P* și *ni* din aceste intervale.

Trebuie încărcată pe Moodle o arhivă care să conțină **în rădăcină** sursele, un Makefile cu target-ul *build* și eventual un fișier README.

Atenție! Fișierul README este necesar doar în cazul în care considerați că este nevoie de explicații adiționale în legătură cu implementarea realizată. Indiferent dacă aveți sau nu fișier README, este important să adăugați comentarii în cod.

~~9 - Test practic APD~~

Să se implementeze, **folosind Pthreads**, un program paralel care realizează diferite operații (adunare, înmulțire, minim, maxim, medie) pe un vector cu N elemente, folosind P thread-uri. Rezolvarea se va face maximizând concurența thread-urilor. Rezultatul se va printa de către un thread ales de către programator (poate fi thread-ul cu un anumit ID), dar **NU** de către thread-ul *main*. Mai mult, doar acestui thread îi va fi permis să citească valorile calculate de celelalte thread-uri.

Atenție! Thread-ul *main* nu are voie să afișeze nimic. Nerespectarea acestei cerințe va duce la un punctaj de **0** pe problemă.

În urma compilării, va rezulta un executabil numit *testapd*, care se va rula astfel: **`./testapd N P n1 n2 ... nN`** (n1…nN sunt cele N numere din vector). În funcție de valoarea lui P, programul va trebui să realizeze una sau mai multe din operațiile de adunare, înmulțire, minim, maxim, medie (în această ordine) pe toate elementele din vector. Astfel, dacă P este 1, se va realiza doar adunare; dacă P este 2, se vor efectua operațiile de adunare și înmulțire pe toate elementele din vector, etc. Fiecare thread va fi responsabil de câte una singură din aceste operații. Aveți mai jos trei exemple de rulare:

```
$ ./testapd 10 3 2 6 3 13 21 11 14 1 17 8
sum: 96
prod: 205837632
min: 1

$ ./testapd 3 2 2 10 4
sum: 16
prod: 80

$ ./testapd 5 5 25 32 11 4 18
sum: 90
prod: 633600
min: 4
max: 32
avg: 18
```

În primul exemplu de mai sus, vectorul este [2,6,3,13,21,11,14,1,17,8] și se rulează cu 3 thread-uri, deci se cer operațiile de adunare, înmulțire și minim pe elementele vectorului, iar rezultatele se afișează în formatul prezentat.

Atenție! Toate thread-urile trebuie pornite în același timp (într-o buclă comună). Programul submis nu trebuie să printeze altceva decât rezultatele cerute. Afișarea se va face dintr-unul din thread-urile pornite de către voi.

Atenție! Un thread realizează o singură operație pe vector. Astfel, în primul exemplu de mai sus, thread-ul 0 calculează suma elementelor vectorului, thread-ul 1 produsul, thread-ul 2 minimul, și unul singur din ele afișează rezultatele celor trei operații.

Se presupune că parametrii de intrare sunt corecți (atât ca număr și ordine, cât și ca valori). Valorile acceptate pentru *N*, *P* și *ni* sunt: $0 < N <= 100$, $0 < P <= 5$, $0 <= ni < 100$. Programul trebuie să funcționeze corect pentru orice combinație de *N*, *P* și *ni* din aceste intervale.

Trebuie încărcată pe Moodle o arhivă care să conțină **în rădăcină** sursele, un Makefile cu target-ul *build* și eventual un fișier README.

Atenție! Fișierul README este necesar doar în cazul în care considerați că este nevoie de explicații adiționale în legătură cu implementarea realizată. Indiferent dacă aveți sau nu fișier README, este important să adăugați comentarii în cod.

JAVA

~~10 - Test practic APD~~

Să se implementeze, **folosind Java Threads**, un algoritm paralel pentru determinarea numărului de apariții ale unei valori într-un vector cu N elemente, precum și a sumei pozițiilor la care valoarea se găsește în vector, folosind P thread-uri. Rezolvarea se va face maximizând concurența thread-urilor. Rezultatul se va printa de către un thread ales de către programator (poate fi thread-ul cu un anumit ID), dar **NU** de către thread-ul *main*. Mai mult, doar acestui thread îi va fi permis să citească valorile parțiale calculate de celelalte thread-uri.

Atenție! Thread-urile nu au voie să incrementeze niciun contor partajat de către toate, deoarece incrementarea lui presupune citirea valorii inițiale.

În urma compilării prin intermediul comenzii `javac *.java`, va rezulta un fișier numit *Testapd.class*, care se va rula astfel: **`java Testapd N P X`** (unde X este valoarea căutată). Vectorul în care se caută valoarea va fi inițializat în funcția *main* ca **`v[i] = (i + 3) % 5`**, înainte de pornirea thread-urilor. Programul va trebui să afișeze numărul de apariții ale elementului căutat în vector, apoi suma pozițiilor aparițiilor, cu spațiu între ele. Aveți mai jos trei exemple de rulare:

```
$ javac *.java
[...]

$ java Testapd 10 3 3
2 5

$ java Testapd 25 2 4
5 55

$ java Testapd 3 8 0
1 2
```

În primul exemplu de mai sus, vectorul este [3,4,0,1,2,3,4,0,1,2], deci elementul căutat (adică 3) se află la pozițiile 0 și 5. În acest caz, numărul de apariții este 2, iar suma acestora este 5.

Atenție! Toate thread-urile trebuie pornite în același timp (într-o buclă comună). Programul submis nu trebuie să printeze altceva decât numărul de apariții și suma pozițiilor aparițiilor, în această ordine și în formatul de mai sus. Afișarea se va face dintr-unul din thread-urile pornite de către voi.

Atenție! Formula cu care se inițializează vectorul este doar pentru a ușura testarea. Nu o folosiți pentru a deduce matematic numărul de apariții. Fiecare thread trebuie să itereze prin fragmentul de vector alocat lui și să găsească aparițiile valorii căutate.

Se presupune că parametrii de intrare sunt corecți (atât ca număr și ordine, cât și ca valori). Valorile acceptate pentru N, P și X sunt: $0 < N <= 10000$, $0 < P <= 8$, $0 <= X < 5$. Programul trebuie să funcționeze corect pentru orice combinație de N, P și X din aceste intervale.

Trebuie să încărcați pe Moodle o arhivă care să conțină **în rădăcină** fișierele sursă și eventual un fișier README.

Atenție! Fișierul README este necesar doar în cazul în care considerați că este nevoie de explicații adiționale în legătură cu implementarea realizată.

~~11 - Test practic APD~~

Să se implementeze, **folosind Java Threads**, un program paralel cu P thread-uri, în care ultimele P-1 thread-uri realizează un job abstract (un *sleep* cu durata între 1 și 7 secunde, așa cum se va specifica mai jos), iar thread-ul 0 afișează mesajul „Thread-ul X a terminat” **imediat** ce fiecare dintre ele a terminat execuția job-ului abstract.

În urma compilării prin intermediul comenzii `javac *.java`, va rezulta un fișier numit *Testapd.class*, care va primi ca parametri valoarea lui P și durata de *sleep* (în secunde) a fiecăruia din ultimele P-1 thread-uri. Aveți mai jos trei exemple de rulare:

```
$ javac *.java
[...]

$ java Testapd 3 2 4
Thread-ul 1 a terminat
Thread-ul 2 a terminat

$ java Testapd 3 4 2
Thread-ul 2 a terminat
Thread-ul 1 a terminat

$ java Testapd 8 7 6 5 4 3 2 1
Thread-ul 7 a terminat
Thread-ul 6 a terminat
Thread-ul 5 a terminat
Thread-ul 4 a terminat
Thread-ul 3 a terminat
Thread-ul 2 a terminat
Thread-ul 1 a terminat
```

În primul exemplu de rulare de mai sus, se specifică faptul că vor fi trei thread-uri. Dintre acestea, thread-ul 1 va executa un job abstract (adică va face un *sleep*) de 2 secunde, iar thread-ul 2 va avea alocat un job de 4 secunde. În momentul în care thread-ul 1 termină de executat job-ul (adică la 2 secunde după pornirea sa), thread-ul 0 afișează mesajul „Thread-ul 1 a terminat”. După încă aproximativ 2 secunde, thread-ul 2 termină și el execuția, moment în care thread-ul 0 afișează „Thread-ul 2 a terminat”.

Atenție! Toate thread-urile trebuie pornite în același timp (într-o buclă comună). Doar thread-ul 0 are voie sa printeze. Programul submis nu trebuie să printeze altceva decât mesajele de tipul „Thread-ul X a terminat” (câte unul pe linie).

Se presupune că parametrii de intrare sunt corecți (atât ca număr și ordine, cât și ca valori). Valorile acceptate pentru P sunt 2 <= P <= 8, iar pentru durata unui job sunt între 1 și 7 secunde (inclusiv).

Atenție! Se va trata și situația în care thread-ul de afișare este lent, iar mai multe alte thread-uri termină simultan, ele trebuind să **nu** se blocheze în așteptarea lui. O soluție care respectă logica de paralelism, dar nu respectă această condiție, va fi depunctată cu 20% din punctaj.

Trebuie să încărcați pe Moodle o arhivă care să conțină **în rădăcină** fișierele sursă și eventual un fișier README.

Atenție! Fișierul README este necesar doar în cazul în care considerați că este nevoie de explicații adiționale în legătură cu implementarea realizată.

~~12 - Test practic APD~~

Să se implementeze, **folosind Java Threads**, un program paralel pentru sortarea unui vector cu N elemente prin intermediul Rank Sort, folosind P thread-uri. Rezolvarea se va face maximizând concurența thread-urilor.

Rezultatul se va printa de către un thread ales de către programator (poate fi thread-ul cu un anumit ID), dar **NU** de către thread-ul *main*. Mai mult, doar acestui thread îi va fi permis să citească fragmentele parțiale de vector sortate de celelalte thread-uri.

În urma compilării prin intermediul comenzii `javac *.java`, va rezulta un fișier numit *Testapd.class*, care va primi ca parametri valorile lui N și P (în această ordine). Vectorul de sortat va fi inițializat în funcția *main* ca `v[i] = N - i + 5`, înainte de pornirea thread-urilor. Aveți mai jos trei exemple de rulare:

```
$ javac *.java
[...]

$ java Testapd 10 3
6 7 8 9 10 11 12 13 14 15

$ java Testapd 25 4
6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30

$ java Testapd 3 8
6 7 8
```

Atenție! Toate thread-urile trebuie pornite în același timp (într-o buclă comună). Programul submis nu trebuie să printeze altceva decât vectorul sortat. Afișarea vectorului sortat se va face dintr-unul din thread-urile pornite de către voi.

Se presupune că parametrii de intrare sunt corecți (atât ca număr și ordine, cât și ca valori). Valorile acceptate pentru N și P sunt: $0 < N <= 10000$, $0 < P <= 8$. Programul trebuie să funcționeze corect pentru orice combinație de N și P din aceste intervale.

Trebuie să încărcați pe Moodle o arhivă care să conțină **în rădăcină** fișierele sursă și eventual un fișier README.

Atenție! Fișierul README este necesar doar în cazul în care considerați că este nevoie de explicații adiționale în legătura cu implementarea realizată.

~~13 - Test practic APD~~

Să se implementeze, **folosind Java Threads**, un program paralel cu N thread-uri în care fiecare thread generează M perechi de numere naturale (K, V) din intervalele [1, 10], respectiv [1, 100000]. La final, programul va afișa **exact** 10 perechi de forma (I, V_max_i) unde V_max_i este maximul valorilor V generate în perechile (I, V) din toate thread-urile.

Valorile M se vor alege aleator între 2 și 10 de către fiecare thread, iar rezultatul trebuie să se printeze o singură dată, la final. Implementarea va maximiza concurența între thread-uri.

Introducerea unei valori în cadrul unui thread va respecta **obligatoriu** următorul pseudocod:

- generează aleator o pereche nouă (K, V)
- printează un mesaj de forma `[Thread %d] a generat perechea (%d, %d)`
- citește valoarea maximă anterioară pentru cheia K
- așteaptă (folosind `sleep`) o secundă
- actualizează valoarea nouă de la cheia K.

În urma compilării prin intermediul comenzii `javac *.java`, va rezulta un fișier numit *Testapd.class*, care se va rula astfel: **`java Testapd N`** (unde N este numărul de thread-uri). Se presupune că programul este apelat corect și trebuie să funcționeze corect pentru orice valoare a lui N între 1 și 8 (inclusiv). Programul va trebui să afișeze mesajul de mai sus de fiecare dată când un thread generează o pereche nouă, precum și rezultatul final. Aveți mai jos un exemplu de rulare:

```
$ .javac *.java

$ java Testapd 3
[Thread 0] a generat perechea (2, 7)
[Thread 1] a generat perechea (1, 10)
[Thread 2] a generat perechea (3, 9)
[Thread 0] a generat perechea (1, 5)
[Thread 1] a generat perechea (3, 11)
[Thread 2] a generat perechea (1, 11)
[Thread 0] a generat perechea (8, 25)
[Thread 1] a generat perechea (2, 10)

(1, 11), (2, 10), (3, 11), (4, 0), (5, 0), (6, 0), (7, 0), (8, 25), (9, 0), (10, 0)
```

În exemplul de mai sus, thread-urile 0 și 1 au ales aleator M=3, iar thread-ul 2 a ales M=2. Așa cum se observă, rezultatul final conține înșiruirea celor 10 chei și valorile maxime din toate thread-urile pentru fiecare din ele. Pentru cheile fără valori (4, 6, 7, 9 și 10 din exemplul de mai sus) se consideră valoarea implicită 0.

Atenție! Nu aveți voie să folosiți ConcurrentHashMap sau ceva similar.

Atenție! Implementarea trebuie să maximizeze paralelismul. În exemplu s-au făcut 8 operații cu durata minimă de câte o secundă, însă datorită ordinii lor, toate thread-urile pot executa (chiar și cu respectarea pseudocodului de adăugare) mai multe operații la fiecare moment, deci timpul rezultat va fi (cel mai probabil) mai mic. O implementare care serializează operațiile nu va fi acceptată.

Trebuie să încărcați pe Moodle o arhivă care să conțină **în rădăcină** fișierele sursă și eventual un fișier README. Fișierul README este necesar doar în cazul în care considerați că este nevoie de explicații adiționale în legătură cu implementarea realizată.

~~14 - Test practic APD~~

Să se implementeze, **folosind Java Threads**, un algoritm paralel pentru determinarea numărului de valori prime dintr-un vector cu N elemente, precum și a sumei pozițiilor la care numerele prime se găsesc în vector, folosind P thread-uri. Rezolvarea se va face maximizând concurența thread-urilor. Rezultatul se va printa de către un thread ales de către programator (poate fi thread-ul cu un anumit ID), dar **NU** de către thread-ul *main*. Mai mult, doar acestui thread îi va fi permis să citească valorile parțiale calculate de celelalte thread-uri.

Atenție! Thread-urile nu au voie să incrementeze niciun contor unic **partajat de către toate**, deoarece incrementarea lui presupune citirea valorii inițiale. Nerespectarea acestei cerințe va duce la un punctaj de **0** pe problemă.

Atenție! Thread-ul *main* nu are voie să afișeze nimic. Nerespectarea acestei cerințe va duce la un punctaj de **0** pe problemă.

În urma compilării prin intermediul comenzii `javac *.java`, va rezulta un fișier numit *Testapd.class*, care se va rula astfel: **`java Testapd N P n1 n2 ... nN`** (n1…nN sunt cele N numere din vector). Programul va trebui să afișeze numărul de elemente prime din vector, apoi suma pozițiilor acestora, cu spațiu între ele. Aveți mai jos trei exemple de rulare:

```
$ javac *.java
[...]

$ java Testapd 10 3 2 6 3 13 21 11 14 1 17 8
5 18

$ java Testapd 3 4 13 14 15
1 0

$ java Testapd 5 2 29 32 11 4 18
2 2
```

În primul exemplu de mai sus, vectorul este [2,6,3,13,21,11,14,1,17,8], deci elementele prime se află la pozițiile 0, 2, 3, 5 și 8. În acest caz, numărul de elemente prime este 5, iar suma pozițiilor acestora este 18.

Atenție! Toate thread-urile trebuie pornite în același timp (într-o buclă comună). Programul submis nu trebuie să printeze altceva decât numărul de elemente prime și suma pozițiilor acestora, în această ordine și în formatul de mai sus. Afișarea se va face dintr-unul din thread-urile pornite de către voi.

Se presupune că parametrii de intrare sunt corecți (atât ca număr și ordine, cât și ca valori). Valorile acceptate pentru *N*, *P* și *ni* sunt: $0 < N <= 100$, $0 < P <= 8$, $0 <= ni < 100$. Programul trebuie să funcționeze corect pentru orice combinație de *N*, *P* și *ni* din aceste intervale.

Trebuie încărcată pe Moodle o arhivă care să conțină **în rădăcină** sursele și eventual un fișier README.

Atenție! Fișierul README este necesar doar în cazul în care considerați că este nevoie de explicații adiționale în legătură cu implementarea realizată. Indiferent dacă aveți sau nu fișier README, este important să adăugați comentarii în cod.

~~15 - Test practic APD~~

Să se implementeze, **folosind Java Threads**, un program paralel pentru realizarea operației de Reduce (pentru sumă, produs, minim și maxim) asupra unui vector cu N elemente, folosind P thread-uri. Rezolvarea se va face maximizând concurența thread-urilor. Rezultatul se va printa de către un thread ales de către programator (poate fi thread-ul cu un anumit ID), dar **NU** de către thread-ul *main*. Mai mult, doar acestui thread îi va fi permis să citească valorile parțiale calculate de celelalte thread-uri.

Atenție! Thread-urile nu au voie să acumuleze într-un contor unic **partajat de către toate**, deoarece modificarea lui presupune citirea valorii inițiale. Nerespectarea acestei cerințe va duce la un punctaj de **0** pe problemă.

Atenție! Thread-ul *main* nu are voie să afișeze nimic. Nerespectarea acestei cerințe va duce la un punctaj de **0** pe problemă.

În urma compilării prin intermediul comenzii `javac *.java`, va rezulta un fișier numit *Testapd.class*, care se va rula astfel: **`java Testapd N P Operatie n1 n2 ... nN`** (al treilea parametru reprezintă operația aplicată asupra tuturor elementelor, iar n1…nN sunt cele N numere din vector). Programul va trebui să afișeze rezultatul aplicării operației asupra tuturor elementelor din vector. Aveți mai jos trei exemple de rulare:

```
$ javac *.java
[...]

$ java Testapd 10 3 sum 2 6 3 13 21 11 14 1 17 8
96

$ java Testapd 3 4 prod 2 10 4
80

$ java Testapd 5 2 min 29 32 11 4 18
4
```

În primul exemplu de mai sus, vectorul este [2,6,3,13,21,11,14,1,17,8] și operația dorită este suma, deci rezultatul afișat va fi 96.

Atenție! Toate thread-urile trebuie pornite în același timp (într-o buclă comună). Programul submis nu trebuie să printeze altceva decât rezultatul operației cerute asupra tuturor elementelor din vector. Afișarea se va face dintr-unul din thread-urile pornite de către voi.

Se presupune că parametrii de intrare sunt corecți (atât ca număr și ordine, cât și ca valori). Valorile acceptate pentru *N*, *P* și *ni* sunt: 0 < *N* <= 100, 0 < *P* <= 8, 0 <= *ni* < 100. Operația poate fi *sum*, *prod*, *min*, sau *max*. Programul trebuie să funcționeze corect pentru orice combinație de *N*, *P* și *ni* din aceste intervale, pentru orice operație.

Trebuie încărcată pe Moodle o arhivă care să conțină **în rădăcină** sursele și eventual un fișier README.

Atenție! Fișierul README este necesar doar în cazul în care considerați că este nevoie de explicații adiționale în legătură cu implementarea realizată. Indiferent dacă aveți sau nu fișier README, este important să adăugați comentarii în cod.

16 - Test practic APD

Să se implementeze, **folosind Java Threads**, un program paralel pentru realizarea operației de Scan (pentru sumă) asupra unui vector cu N elemente, folosind P thread-uri. Rezolvarea se va face maximizând concurența thread-urilor. Rezultatul se va printa de către un thread ales de către programator (poate fi thread-ul cu un anumit ID), dar **NU** de către thread-ul *main*.

Atenție! Thread-ul *main* nu are voie să afișeze nimic. Nerespectarea acestei cerințe va duce la un punctaj de **0** pe problemă.

În urma compilării prin intermediul comenzii `javac *.java`, va rezulta un fișier numit *Testapd.class*, care se va rula astfel: **`java Testapd N P n1 n2 ... nN`** (n1…nN sunt cele N numere din vector). Programul va trebui să afișeze sumele parțiale, în ordine. Aveți mai jos trei exemple de rulare:

```
$ javac *.java
[...]

$ java Testapd 10 3 2 6 3 13 21 11 14 1 17 8
2 8 11 24 45 56 70 71 88 96

$ java Testapd 3 4 2 10 4
2 12 16

$ java Testapd 5 2 29 32 11 4 18
29 61 72 76 94
```

În primul exemplu de mai sus, vectorul este [2,6,3,13,21,11,14,1,17,8], deci se afișează toate sumele parțiale în ordine (primul element, suma primelor două elemente, suma primelor trei elemente, etc.).

Atenție! Toate thread-urile trebuie pornite în același timp (într-o buclă comună). Programul submis nu trebuie să printeze altceva decât rezultatele parțiale cerute. Afișarea se va face dintr-unul din thread-urile pornite de către voi.

Se presupune că parametrii de intrare sunt corecți (atât ca număr și ordine, cât și ca valori). Valorile acceptate pentru *N*, *P* și *ni* sunt: $0 < N <= 100$, $0 < P <= 8$, $0 <= ni < 100$. Programul trebuie să funcționeze corect pentru orice combinație de *N*, *P* și *ni* din aceste intervale.

Trebuie încărcată pe Moodle o arhivă care să conțină **în rădăcină** sursele și eventual un fișier README.

Atenție! Fișierul README este necesar doar în cazul în care considerați că este nevoie de explicații adiționale în legătură cu implementarea realizată. Indiferent dacă aveți sau nu fișier README, este important să adăugați comentarii în cod.

~~17 - Test practic APD~~

Să se implementeze, **folosind Java Threads**, un program paralel care caută, într-un vector cu N elemente, numerele divizibile cu anumite valori, folosind P thread-uri. Rezolvarea se va face maximizând concurența thread-urilor. Rezultatul se va printa de către un thread ales de către programator (poate fi thread-ul cu un anumit ID), dar **NU** de către thread-ul *main*. Mai mult, doar acestui thread îi va fi permis să citească valorile calculate de celelalte thread-uri.

Atenție! Thread-ul *main* nu are voie să afișeze nimic. Nerespectarea acestei cerințe va duce la un punctaj de **0** pe problemă.

În urma compilării prin intermediul comenzii `javac *.java`, va rezulta un fișier numit *Testapd.class*, care se va rula astfel: **`java Testapd N P n1 n2 ... nN`** (n1…nN sunt cele N numere din vector). Programul va trebui să afișeze toate numerele divizibile cu 2, …, P+1, iar un thread cu ID-ul ID va fi responsabil de verificarea divizibilității pentru divizorul ID+2. Aveți mai jos trei exemple de rulare:

```
$ javac *.java
[...]

$ java Testapd 10 3 2 6 3 13 21 11 14 1 17 8
2: 2 6 14 8
3: 6 3 21
4: 8

$ java Testapd 3 2 2 10 4
2: 2 10 4
3:

$ java Testapd 5 4 25 32 11 4 18
2: 32 4 18
3: 18
4: 32 4
5: 25
```

În primul exemplu de mai sus, vectorul este [2,6,3,13,21,11,14,1,17,8] și se rulează cu 3 thread-uri, deci se vor căuta numerele divizibile cu 2, 3 și 4, care se afișează în formatul prezentat.

Atenție! Toate thread-urile trebuie pornite în același timp (într-o buclă comună). Programul submis nu trebuie să printeze altceva decât rezultatele cerute. Afișarea se va face dintr-unul din thread-urile pornite de către voi.

Atenție! Doar thread-ul cu ID-ul ID trebuie să verifice ce numere sunt divizibile cu ID+2. Astfel, în primul exemplu de mai sus, thread-ul 0 verifică numerele divizibile cu 2, thread-ul 1 pe cele divizibile cu 3, și thread-ul 2 pe cele divizibile cu 4, iar unul singur din cele trei thread-uri face afișarea.

Se presupune că parametrii de intrare sunt corecți (atât ca număr și ordine, cât și ca valori). Valorile acceptate pentru *N*, *P* și *ni* sunt: $0 < N <= 100$, $0 < P <= 8$, $0 <= ni < 100$. Programul trebuie să funcționeze corect pentru orice combinație de *N*, *P* și *ni* din aceste intervale.

Trebuie încărcată pe Moodle o arhivă care să conțină **în rădăcină** sursele și eventual un fișier README.

Atenție! Fișierul README este necesar doar în cazul în care considerați că este nevoie de explicații adiționale în legătură cu implementarea realizată. Indiferent dacă aveți sau nu fișier README, este important să adăugați comentarii în cod.

~~18 - Test practic APD~~

Să se implementeze, **folosind Java Threads**, un program paralel care realizează diferite operații (adunare, înmulțire, minim, maxim, medie) pe un vector cu N elemente, folosind P thread-uri. Rezolvarea se va face maximizând concurența thread-urilor. Rezultatul se va printa de către un thread ales de către programator (poate fi thread-ul cu un anumit ID), dar **NU** de către thread-ul *main*. Mai mult, doar acestui thread îi va fi permis să citească valorile calculate de celelalte thread-uri.

Atenție! Thread-ul *main* nu are voie să afișeze nimic. Nerespectarea acestei cerințe va duce la un punctaj de **0** pe problemă.

În urma compilării prin intermediul comenzii `javac *.java`, va rezulta un fișier numit *Testapd.class*, care se va rula astfel: **`java Testapd N P n1 n2 ... nN`** (n1…nN sunt cele N numere din vector). În funcție de valoarea lui P, programul va trebui să realizeze una sau mai multe din operațiile de adunare, înmulțire, minim, maxim, medie (în această ordine) pe toate elementele din vector. Astfel, dacă P este 1, se va realiza doar adunare; dacă P este 2, se vor efectua operațiile de adunare și înmulțire pe toate elementele din vector, etc. Fiecare thread va fi responsabil de câte una singură din aceste operații. Aveți mai jos trei exemple de rulare:

```
$ javac *.java
[...]

$ java Testapd 10 3 2 6 3 13 21 11 14 1 17 8
sum: 96
prod: 205837632
min: 1

$ java Testapd 3 2 2 10 4
sum: 16
prod: 80

$ java Testapd 5 5 25 32 11 4 18
sum: 90
prod: 633600
min: 4
max: 32
avg: 18
```

În primul exemplu de mai sus, vectorul este [2,6,3,13,21,11,14,1,17,8] și se rulează cu 3 thread-uri, deci se cer operațiile de adunare, înmulțire și minim pe elementele vectorului, iar rezultatele se afișează în formatul prezentat.

Atenție! Toate thread-urile trebuie pornite în același timp (într-o buclă comună). Programul submis nu trebuie să printeze altceva decât rezultatele cerute. Afișarea se va face dintr-unul din thread-urile pornite de către voi.

Atenție! Un thread realizează o singură operație pe vector. Astfel, în primul exemplu de mai sus, thread-ul 0 calculează suma elementelor vectorului, thread-ul 1 produsul, thread-ul 2 minimul, și unul singur din ele afișează rezultatele celor trei operații.

Se presupune că parametrii de intrare sunt corecți (atât ca număr și ordine, cât și ca valori). Valorile acceptate pentru *N*, *P* și *ni* sunt: 0 < *N* <= 100, 0 < *P* <= 5, 0 <= *ni* < 100. Programul trebuie să funcționeze corect pentru orice combinație de *N*, *P* și *ni* din aceste intervale.

Trebuie încărcată pe Moodle o arhivă care să conțină **în rădăcină** sursele și eventual un fișier README.

Atenție! Fișierul README este necesar doar în cazul în care considerați că este nevoie de explicații adiționale în legătură cu implementarea realizată. Indiferent dacă aveți sau nu fișier README, este important să adăugați comentarii în cod.

MPI

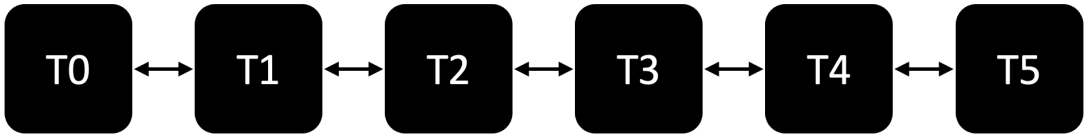
~~19 - Test practic APD~~

Testul practic este format din două cerințe care se punctează individual, cu mențiunea că a doua cerință este dependentă de rezolvarea primei cerințe.

Cerința 1 (6p). Să se implementeze, **folosind C/C++ și MPI** și presupunând **o topologie cu N task-uri conectate în linie** crescător după rang, algoritmul de sortare OETS pentru N elemente. Astfel, **fiecare task MPI va conține inițial câte un număr**. La finalul execuției programului vostru, valorile vor fi sortate **crescător**: task-ul cu rangul 0 va avea numărul cel mai mic, task-ul cu rangul 1 va avea următorul număr, etc. După rularea algoritmului de sortare, fiecare task își va afișa rangul și valoarea sa finală (pe formatul *rang - valoare*).

Cerința 2 (4p). Odată ce numerele au fost sortate corect folosind OETS, se cere colectarea valorilor la **task-ul 0** și afișarea acestora în ordine **crescătoare** (fără a se mai sorta o dată valorile venite). Comunicarea se poate realiza doar între task-uri vecine (ca și la cerința 1).

Puteți observa în imaginea de mai jos un exemplu de topologie pentru 6 task-uri, unde, de exemplu, task-ul 0 poate să comunice direct doar cu task-ul 1, task-ul 1 poate sa comunice direct doar cu task-urile 0 și 2, etc.



Algoritmul OETS este prezentat în laboratorul 3 și are următorul pseudocod pentru situația în care fiecare task are câte un număr:

```
for i = [0, n)
    if (i % 2 == 0)
        task-urile pare compara valoarea cu vecinul drept si pastreaza minimul
        task-urile impare compara valoarea cu vecinul stang si pastreaza maximul
    else
        task-urile impare compara valoarea cu vecinul drept si pastreaza minimul
        task-urile pare compara valoarea cu vecinul stang si pastreaza maximul
```

În urma compilării, va rezulta un executabil numit *testapd*, care se va putea rula prin intermediul comenzii *mpirun -np N ./testapd*. Fiecare task va avea valoarea inițială setată la **2N - 2R** (unde N este numărul de task-uri, iar R este rangul task-ului curent). Aveți mai jos trei exemple de rulare, unde output-ul cu albastru este cel necesar pentru prima cerință, iar cel cu roșu este specific pentru a doua cerință. Afișarea trebuie realizată pe formatul prezentat.

```
$ mpirun -np 5 ./testapd
3 - 8
1 - 4
0 - 2
4 - 10
2 - 6
2 4 6 8 10
```

```
$ mpirun -np 6 ./testapd
0 - 2
1 - 4
3 - 8
2 - 6
5 - 12
4 - 10
2 4 6 8 10 12

$ mpirun -np 2 ./testapd
1 - 4
0 - 2
2 4
```

Atenție! Pentru prima cerință, este suficient să afișați doar liniile cu albastru, din fiecare task (deci ordinea afișării lor pe ecran nu va fi neapărat aceeași ca în exemplele de mai sus). Pentru a doua cerință, doar task-ul 0 are voie să afișeze.

Atenție! Comunicația se va face strict cu task-urile vecine (cel de dinainte și cel de după task-ul curent, adică R - 1 și R + 1, dacă există). Dacă se realizează comunicație între task-uri neconectate în topologie la vreuna din cerințe, punctajul pe acea cerință va fi 0.

Atenție! O soluție care nu folosește ideea de la OETS (interschimbări între elemente de pe poziții cu paritate diferită) sau care nu respectă alte specificații ale enunțului (cum ar fi N task-uri pentru N elemente) NU va fi punctată.

Atenție! La rezolvarea cerinței 1, fiecare task va memora cel mult 2 valori la un moment dat.

Programul trebuie să funcționeze corect pentru orice valoare a lui N care respectă 2 <= N <= 12.

Trebuie să încărcați pe Moodle o arhivă care să conțină **în rădăcină** fișierele sursă, un Makefile cu target-ul *build* și eventual un fișier README.

Atenție! Fișierul README este necesar doar în cazul în care considerați că este nevoie de explicații adiționale în legătura cu implementarea realizată.

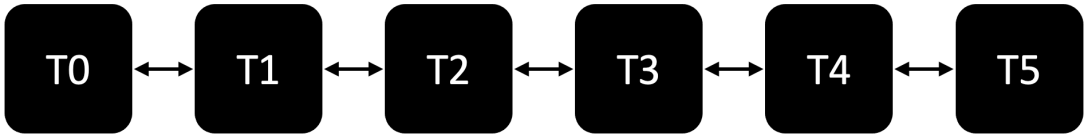
~~20 - Test practic APD~~

Testul practic este format din două cerințe care se punctează individual, cu mențiunea că a doua cerință este dependentă de rezolvarea primei cerințe.

Cerința 1 (6p). Să se implementeze, **folosind C/C++ și MPI** și presupunând **o topologie cu N task-uri conectate în linie** crescător după rang, algoritmul de sortare OETS pentru N elemente. Astfel, **fiecare task MPI va conține inițial câte un număr**. La finalul execuției programului vostru, valorile vor fi sortate **descrescător**: task-ul cu rangul 0 va avea numărul cel mai mare, task-ul cu rangul 1 va avea următorul număr, etc. După rularea algoritmului de sortare, fiecare task își va afișa rangul și valoarea sa finală (pe formatul *rang - valoare*).

Cerința 2 (4p). Odată ce numerele au fost sortate corect folosind OETS, se cere colectarea valorilor la **task-ul N - 1** și afișarea acestora în ordine **descrescătoare** (fără a se mai sorta o dată valorile venite). Comunicarea se poate realiza doar între task-uri vecine.

Puteți observa în imaginea de mai jos un exemplu de topologie pentru 6 task-uri, unde, de exemplu, task-ul 0 poate să comunice direct doar cu task-ul 1, task-ul 1 poate sa comunice direct doar cu task-urile 0 și 2, etc.



Algoritmul OETS este prezentat în laboratorul 3 și are următorul pseudocod pentru situația în care fiecare task are câte un număr:

```
for i = [0, n)
    if (i % 2 == 0)
        task-urile pare compara valoarea cu vecinul drept si pastreaza maximul
        task-urile impare compara valoarea cu vecinul stang si pastreaza minimul
    else
        task-urile impare compara valoarea cu vecinul drept si pastreaza maximul
        task-urile pare compara valoarea cu vecinul stang si pastreaza minimul
```

În urma compilării, va rezulta un executabil numit *testapd*, care se va putea rula prin intermediul comenzii *mpirun -np N ./testapd*. Fiecare task va avea valoarea inițială setată la **2R** (unde R este rangul task-ului curent). Aveți mai jos trei exemple de rulare, unde output-ul cu albastru este cel necesar pentru prima cerință, iar cel cu roșu este specific pentru a doua cerință. Afișarea trebuie realizată pe formatul prezentat.

```
$ mpirun -np 5 ./testapd
3 - 2
1 - 6
0 - 8
4 - 0
2 - 4
8 6 4 2 0
```

```
$ mpirun -np 6 ./testapd
0 - 10
1 - 8
3 - 4
2 - 6
5 - 0
4 - 2
10 8 6 4 2 0

$ mpirun -np 2 ./testapd
1 - 0
0 - 2
2 0
```

Atenție! Pentru prima cerință, este suficient să afișați doar liniile cu albastru, din fiecare task (deci ordinea afișării lor pe ecran nu va fi neapărat aceeași ca în exemplele de mai sus). Pentru a doua cerință, doar task-ul N - 1 are voie să afișeze.

Atenție! Comunicația se va face strict cu task-urile vecine (cel de dinainte și cel de după task-ul curent, adică R - 1 și R + 1, dacă există). Dacă se realizează comunicație între task-uri neconectate în topologie la vreuna din cerințe, punctajul pe acea cerință va fi 0.

Atenție! O soluție care nu folosește ideea de la OETS (interschimbări între elemente de pe poziții cu paritate diferită) sau care nu respectă alte specificații ale enunțului (cum ar fi N task-uri pentru N elemente) NU va fi punctată.

Atenție! La rezolvarea cerinței 1, fiecare task va memora cel mult 2 valori la un moment dat.

Programul trebuie să funcționeze corect pentru orice valoare a lui N care respectă 2 <= N <= 12.

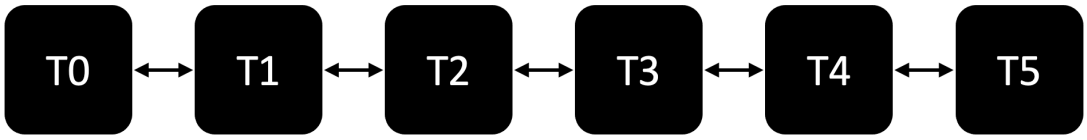
Trebuie să încărcați pe Moodle o arhivă care să conțină **în rădăcină** fișierele sursă, un Makefile cu target-ul *build* și eventual un fișier README.

Atenție! Fișierul README este necesar doar în cazul în care considerați că este nevoie de explicații adiționale în legătura cu implementarea realizată.

~~21 - Test practic APD~~

Să se implementeze, **folosind C/C++ și MPI** și presupunând **o topologie cu N task-uri conectate în linie** crescător după rang, un pipeline care să găsească numerele prime dintr-un șir de N elemente. La final, se cere afișarea elementelor prime, **în ordinea în care apar în șir**, de către **task-ul cu rangul N - 1**. Comunicarea se poate realiza doar între task-uri vecine. Noțiunea de pipeline este prezentată în laboratorul 9.

Puteți observa în imaginea de mai jos un exemplu de topologie pentru 6 task-uri, unde, de exemplu, task-ul 0 poate să comunice direct doar cu task-ul 1, task-ul 1 poate sa comunice direct doar cu task-urile 0 și 2, etc.



În urma compilării, va rezulta un executabil numit *testapd*, care se va putea rula prin intermediul comenzii *mpirun -np N ./testapd nr1 nr2 nr3 […] nrN*. Astfel, programul va primi ca argumente cele N numere din șirul în care se caută valorile prime, **pe care doar task-ul cu rangul 0 are voie sa le citească**. Aveți mai jos trei exemple de rulare, unde output-ul este afișat de către task-ul cu rangul N - 1. Afișarea trebuie realizată pe formatul prezentat, doar de către task-ul N-1.

```
$ mpirun -np 5 ./testapd 17 25 113 22 8
17 113

$ mpirun -np 6 ./testapd 59 37 22 81 89 2
59 37 89 2

$ mpirun -np 2 ./testapd 10 100
Nu exista numere prime in sir
```

Atenție! Fiecare task are voie să facă **o singură** verificare că un număr este prim. O soluție în care un task face mai multe astfel de verificări (fie pentru același număr, fie pentru numere diferite) se va puncta cu 0.

Atenție! Doar task-ul cu rangul 0 are voie să citească argumentele programului din *argv* și doar task-ul N-1 are voie să afișeze.

Atenție! Comunicația se va face strict cu task-urile vecine (cel de dinainte și cel de după task-ul curent cu rangul R, adică R - 1 și R + 1, dacă există). Dacă se realizează comunicație între task-uri neconectate în topologie, punctajul va fi 0.

Atenție! Nerespectarea oricărei restricții de la punctele (de Atenție!) anterioare va face ca soluția să fie notată cu 0 (zero).

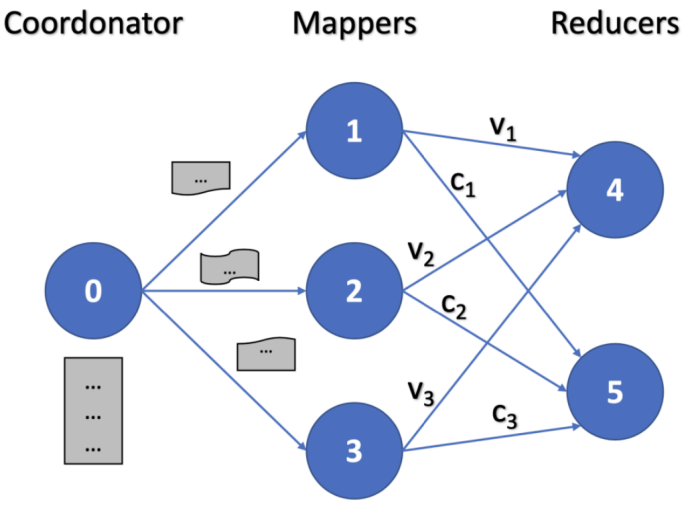
Programul trebuie să funcționeze corect pentru orice valoare a lui N care respectă 2 <= N <= 12. Se poate presupune că numărul de argumente date programului este corect.

Trebuie să încărcați pe Moodle o arhivă care să conțină **în rădăcină** fișierele sursă, un Makefile cu target-ul *build* și eventual un fișier README.

Atenție! Fișierul README este necesar doar în cazul în care considerați că este nevoie de explicații adiționale în legătură cu implementarea realizată.

22 - Test practic APD

Implementați paradigma Map-Reduce în MPI pentru a număra vocalele și consoanele dintr-un fișier. Se presupune că avem o topologie de **exact** 6 task-uri, în care task-ul 0 este coordonatorul, task-urile 1, 2 și 3 au rol de Mappers, iar task-urile 4 și 5 sunt Reducers, conform diagramei de mai jos.



Pentru programul implementat, se va folosi un fișier de intrare cu formatul următor:

```
$ cat file.in
10
hsBoDIVH
FuLHnOJjCw
qtSOewR
SMKigK
HNgyFXgN
EBzHIahNk
CnvBJczolE
domvUzJg
CChCCdkN
AGcDtrfpM
```

În fișierul de mai sus, prima linie conține numărul N de rânduri de analizat, iar apoi urmează cele N rânduri, care pot avea dimensiuni diferite (maxim 15 caractere per rând).

Programul trebuie să urmeze pașii de mai jos:

1. task-ul 0 citește fișierul de intrare și trimite (în mod cât mai echilibrat) rândurile de analizat către cei trei Mappers
2. task-urile de tip Mapper primesc rândurile alocate lor și numără vocalele și consoanele din ele
3. când un Mapper își termină treaba, trimite numărul de vocale task-ului 4 (primul Reducer) și numărul de consoane task-ului 5 (al doilea Reducer)
4. task-ul 4 primește numărul de vocale de la fiecare Mapper, le adună și apoi afișează valoarea finală

5. task-ul 5 primește numărul de consoane de la fiecare Mapper, le adună și apoi afișează valoarea finală.

În urma compilării, va rezulta un executabil numit *testapd*, care se va putea rula prin intermediul comenzii *mpirun -np 6 ./testapd file.in*. Un exemplu de output pentru fișierul de intrare de mai sus ar putea fi următorul (unde se poate observa rangul procesului care printează între parantezele pătrate):

```
$ mpirun -np 6 ./testapd file.in
[1] hsBoDIVH
[1] FuLHnOJjCw
[1] qtSOewR
[1] v=6 c=19
[2] SMKigK
[2] HNgyFXgN
[2] EBzHIahNk
[2] v=4 c=19
[3] CnvBJczolE
[3] domvUzJg
[3] CChCCdkN
[3] AGcDtrfpM
[3] v=5 c=30
[4] Vowels: 15
[5] Consonants: 68
```

Atenție! Pentru notă maximă, programul trebuie să funcționeze corect pentru orice număr de linii în fișierul de intrare. De asemenea, nu trebuie să permită rularea cu un număr de task-uri diferit de 6.

Puteți găsi o descriere a paradigmei Map-Reduce aici.

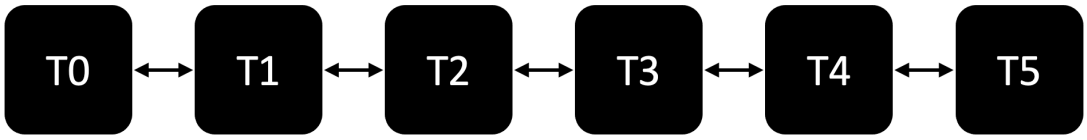
Trebuie să încărcați pe Moodle o arhivă care să conțină **în rădăcină** fișierele sursă, un Makefile cu target-ul *build* și eventual un fișier README.

Atenție! Fișierul README este necesar doar în cazul în care considerați că este nevoie de explicații adiționale în legătura cu implementarea realizată.

23 - Test practic APD

Să se implementeze, **folosind C/C++ și MPI** și presupunând **o topologie cu N task-uri conectate în linie** crescător după rang, un pipeline care să sorteze crescător numerele dintr-un șir de N elemente în funcție de numărul de cifre de 1 pe care le conțin. La final, se cere afișarea elementelor **sortate**, de către **task-ul cu rangul N - 1**. Comunicarea se poate realiza doar între task-uri vecine. Noțiunea de pipeline este prezentată în laboratorul 9.

Puteți observa în imaginea de mai jos un exemplu de topologie pentru 6 task-uri, unde, de exemplu, task-ul 0 poate să comunice direct doar cu task-ul 1, task-ul 1 poate sa comunice direct doar cu task-urile 0 și 2, etc.



În urma compilării, va rezulta un executabil numit *testapd*, care se va putea rula prin intermediul comenzii *mpirun -np N ./testapd nr1 nr2 nr3 […] nrN*. Astfel, programul va primi ca argumente cele N numere din șirul care se va sorta, **pe care doar task-ul cu rangul 0 are voie sa le citească**. Aveți mai jos trei exemple de rulare, unde output-ul este afișat de către task-ul cu rangul N - 1. Afișarea trebuie realizată pe formatul prezentat, doar de către task-ul N - 1.

```
$ mpirun -np 5 ./testapd 17 210 113 22 8
8 22 17 210 113

$ mpirun -np 6 ./testapd 11 31 22 81 151 111
22 31 81 11 151 111

$ mpirun -np 2 ./testapd 10 101
10 101
```

Dacă două valori au același număr de cifre de 1, se vor sorta în funcție de ordinea naturală. Astfel, în primul exemplu din cele de mai sus, 8 și 22 nu au cifre de 1, dar 8 este mai mic decât 22, deci apare primul. Similar, 17 și 210 au câte o cifră de 1, dar 17 este mai mic decât 210.

Atenție! Doar task-ul cu rangul 0 are voie să citească argumentele programului din *argv* și doar task-ul N - 1 are voie să afișeze.

Atenție! Comunicația se va face strict cu task-urile vecine (cel de dinainte și cel de după task-ul curent cu rangul R, adică R - 1 și R + 1, dacă există).

Atenție! Nerespectarea oricărei restricții de la punctele (de Atenție!) anterioare va face ca soluția să fie notată cu 0 (zero).

Programul trebuie să funcționeze corect pentru orice valoare a lui N care respectă 2 <= N <= 12. Se poate presupune că numărul de argumente date programului este corect.

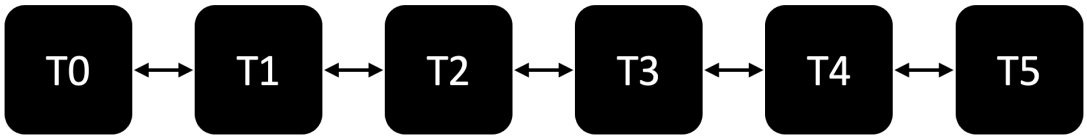
Trebuie să încărcați pe Moodle o arhivă care să conțină **în rădăcină** fișierele sursă, un Makefile cu target-ul *build* și eventual un fișier README.

Atenție! Fișierul README este necesar doar în cazul în care considerați că este nevoie de explicații adiționale în legătură cu implementarea realizată. Indiferent dacă aveți sau nu fișier README, este important să adăugați comentarii în cod.

24 - Test practic APD

Să se implementeze, **folosind C/C++ și MPI** și presupunând **o topologie cu N task-uri conectate în linie** crescător după rang, un pipeline care să sorteze descrescător numerele dintr-un șir de N elemente în funcție de numărul de cifre pare pe care le conțin. La final, se cere afișarea elementelor **sortate**, de către **task-ul cu rangul N - 1**. Comunicarea se poate realiza doar între task-uri vecine. Noțiunea de pipeline este prezentată în laboratorul 9.

Puteți observa în imaginea de mai jos un exemplu de topologie pentru 6 task-uri, unde, de exemplu, task-ul 0 poate să comunice direct doar cu task-ul 1, task-ul 1 poate sa comunice direct doar cu task-urile 0 și 2, etc.



În urma compilării, va rezulta un executabil numit *testapd*, care se va putea rula prin intermediul comenzii *mpirun -np N ./testapd nr1 nr2 nr3 […] nrN*. Astfel, programul va primi ca argumente cele N numere din șirul care se va sorta, **pe care doar task-ul cu rangul 0 are voie sa le citească**. Aveți mai jos trei exemple de rulare, unde output-ul este afișat de către task-ul cu rangul N - 1. Afișarea trebuie realizată pe formatul prezentat, doar de către task-ul N - 1.

```
$ mpirun -np 5 ./testapd 17 210 113 2210 8
2210 210 8 113 17

$ mpirun -np 6 ./testapd 1106 31521 22 81 2 122
1106 122 22 31521 81 2

$ mpirun -np 2 ./testapd 10 101
101 10
```

Dacă două valori au același număr de cifre pare, se vor sorta (tot descrescător) în funcție de ordinea naturală. Astfel, în primul exemplu din cele de mai sus, 113 și 17 nu au cifre pare, dar 113 este mai mare decât 17, deci apare primul.

Atenție! Doar task-ul cu rangul 0 are voie să citească argumentele programului din *argv* și doar task-ul N - 1 are voie să afișeze.

Atenție! Comunicația se va face strict cu task-urile vecine (cel de dinainte și cel de după task-ul curent cu rangul R, adică R - 1 și R + 1, dacă există).

Atenție! Nerespectarea oricărei restricții de la punctele (de Atenție!) anterioare va face ca soluția să fie notată cu 0 (zero).

Programul trebuie să funcționeze corect pentru orice valoare a lui N care respectă 2 <= N <= 12. Se poate presupune că numărul de argumente date programului este corect.

Trebuie să încărcați pe Moodle o arhivă care să conțină **în rădăcină** fișierele sursă, un Makefile cu target-ul *build* și eventual un fișier README.

Atenție! Fișierul README este necesar doar în cazul în care considerați că este nevoie de explicații adiționale în legătură cu implementarea realizată. Indiferent dacă aveți sau nu fișier README, este important să adăugați comentarii în cod.

25 - Test practic APD

Să se scrie, **folosind C/C++ și MPI**, un program care să implementeze problema producător-consumator. Se presupune existența unui singur producător și a unui singur consumator (reprezentate ca două task-uri MPI), cu un buffer partajat de dimensiune B, și un „total de N elemente care vor fi produse și care trebuie consumate în ordine. Programul trebuie să respecte cele trei criterii principale ale problemei: nu se poate produce în buffer în timp ce se consumă, un producător care vrea să scrie în buffer când acesta este plin se va bloca, iar un consumator care vrea să citească din buffer atunci când acesta este gol se va bloca. Problema producător-consumator este prezentată în laboratorul 5.

În urma compilării, va rezulta un executabil numit *testapd*, care se va putea rula prin intermediul comenzii *mpirun -np 2 ./testapd N B*. Astfel, programul va primi ca argumente numărul de elemente care se vor produce și dimensiunea buffer-ului. Elementele produse vor fi întregi cu valorile 0, …, N-1 (în această ordine). Programul trebuie să afișeze mesaje atunci când se produce un element și atunci când se consumă. Aveți mai jos trei exemple de rulare.

```
$ mpirun -np 2 ./testapd 5 2
[P] S-a produs elementul 0
[P] S-a produs elementul 1
[C] S-a consumat elementul 0
[P] S-a produs elementul 2
[C] S-a consumat elementul 1
[P] S-a produs elementul 3
[C] S-a consumat elementul 2
[P] S-a produs elementul 4
[C] S-a consumat elementul 3
[C] S-a consumat elementul 4

$ mpirun -np 2 ./testapd 3 5
[P] S-a produs elementul 0
[P] S-a produs elementul 1
[P] S-a produs elementul 2
[C] S-a consumat elementul 0
[C] S-a consumat elementul 1
[C] S-a consumat elementul 2

$ mpirun -np 2 ./testapd 4 1
[P] S-a produs elementul 0
[C] S-a consumat elementul 0
[P] S-a produs elementul 1
[C] S-a consumat elementul 1
[P] S-a produs elementul 2
[C] S-a consumat elementul 2
[P] S-a produs elementul 3
[C] S-a consumat elementul 3
```

Atenție! Dacă nu se respectă toate restricțiile (de exemplu, un producător poate să producă deși buffer-ul este plin), soluția va fi notată cu 0.

Programul trebuie să funcționeze corect pentru orice valori N și B care respectă 1 <= N <= 100, 1 <= B <= 100. Se poate presupune că numărul de argumente date programului este corect.

Trebuie să încărcați pe Moodle o arhivă care să conțină **în rădăcină** fișierele sursă, un Makefile cu target-ul *build* și eventual un fișier README.

Atenție! Fișierul README este necesar doar în cazul în care considerați că este nevoie de explicații adiționale în legătură cu implementarea realizată. Indiferent dacă aveți sau nu fișier README, este important să adăugați comentarii în cod.

26 - Test practic APD

Să se scrie, **folosind C/C++ și MPI**, un program care să implementeze paralelizarea algoritmului clasic de înmulțire a două matrice pătratice N x N în MPI, folosind P task-uri. Task-ul cu rangul 0 va genera cele două matrice care se înmulțesc și va fi responsabil de alocarea calculelor către celelalte task-uri, și tot el va afișa rezultatul înmulțirii.

În urma compilării, va rezulta un executabil numit *testapd*, care se va putea rula prin intermediul comenzii *mpirun -np P ./testapd N*. Este sarcina task-ului 0 să genereze matricele inițiale de dimensiune N x N ca fiind **`A[i][j] = i + j`** și **`B[i][j] = i - j`**. Aveți mai jos trei exemple de rulare.

```
$ mpirun -np 2 ./testapd 3
5 2 -1
8 2 -4
11 2 -7

$ mpirun -np 3 ./testapd 4
14 8 2 -4
20 10 0 -10
26 12 -2 -16
32 14 -4 -22

$ mpirun -np 4 ./testapd 2
1 0
2 -1
```

Atenție! Doar task-ul cu rangul 0 are voie să afișeze rezultatul final.

Atenție! Trebuie să trimiteți din task-ul 0 către celelalte task-uri **doar cantitatea de date necesare** (nu mai mult). Task-ul 0 trebuie să participe și el la calculul matricei rezultat.

Atenție! Doar rank-ului 0 îi este permis să folosească formulele de calcul pentru inițializarea matricelor. Celelalte rank-uri vor primi de la acesta datele necesare. Mai mult, este necesar ca fiecare proces cu rank != 0 să primească și să stocheze la un moment dat DOAR datele care îi sunt necesare (nu toate datele de intrare).

Programul trebuie să funcționeze corect pentru orice valori N și P care respectă 1 <= N <= 100, 1 <= P <= 8. Se poate presupune că numărul de argumente date programului este corect.

Atenție! Nerespectarea oricărei restricții de la punctele (de Atenție!) anterioare va face ca soluția să fie notată cu 0 (zero).

Trebuie să încărcați pe Moodle o arhivă care să conțină **în rădăcină** fișierele sursă, un Makefile cu target-ul *build* și eventual un fișier README.

Atenție! Fișierul README este necesar doar în cazul în care considerați că este nevoie de explicații adiționale în legătură cu implementarea realizată. Indiferent dacă aveți sau nu fișier README, este important să adăugați comentarii în cod.

27 - Test practic APD

Să se scrie, **folosind C/C++ și MPI**, un program care să implementeze paralelizarea ridicării la puterea k a unei matrice pătratice N x N în MPI, folosind P task-uri. Task-ul cu rangul 0 va genera matricea care se va ridica la puterea k și va fi responsabil de alocarea calculelor către celelalte task-uri, și tot el va afișa rezultatul final.

În urma compilării, va rezulta un executabil numit *testapd*, care se va putea rula prin intermediul comenzii *mpirun -np P ./testapd N k*. Este sarcina task-ului 0 să genereze matricea inițială de dimensiune N x N ca fiind **`M[i][j] = i + j`**. Aveți mai jos trei exemple de rulare.

```
$ mpirun -np 2 ./testapd 3 4
210 372 534
372 660 948
534 948 1362

$ mpirun -np 3 ./testapd 4 2
14 20 26 32
20 30 40 50
6 40 54 68
32 50 68 86

$ mpirun -np 4 ./testapd 2 3
2 5
5 12
```

Atenție! Doar task-ul cu rangul 0 are voie să afișeze rezultatul final.

Programul trebuie să funcționeze corect pentru orice valori N și P care respectă 1 <= N <= 100, 1 <= P <= 8. Se poate presupune că numărul de argumente date programului este corect.

Atenție! Doar rank-ului 0 îi este permis să folosească formulele de calcul pentru inițializarea matricei. Celelalte rank-uri vor primi de la acesta datele necesare.

Atenție! Nerespectarea oricărei restricții de la punctele (de Atenție!) anterioare va face ca soluția să fie notată cu 0 (zero).

Trebuie să încărcați pe Moodle o arhivă care să conțină **în rădăcină** fișierele sursă, un Makefile cu target-ul *build* și eventual un fișier README.

Atenție! Fișierul README este necesar doar în cazul în care considerați că este nevoie de explicații adiționale în legătură cu implementarea realizată. Indiferent dacă aveți sau nu fișier README, este important să adăugați comentarii în cod.

28 - Test practic APD

Să se implementeze, **folosind C/C++ și MPI**, un program distribuit pentru determinarea numărului de apariții ale unei valori într-un vector cu N elemente, precum și a sumei pozițiilor la care valoarea se găsește în vector, folosind P task-uri. Rezolvarea se va face maximizând concurența task-urilor. Rezultatul se va printa de către un task ales de către programator (poate fi task-ul cu un anumit ID), dar **NU** de către task-ul 0.

În urma compilării, va rezulta un executabil numit *testapd*, care se va rula astfel: **`mpirun -np P ./testapd N X`** (unde X este valoarea căutată).

Vectorul în care se caută valoarea va fi inițializat **doar** de task-ul 0 ca **`v[i] = (i + 3) % 5`**.

Programul va trebui să afișeze numărul de apariții ale elementului căutat în vector, apoi suma pozițiilor aparițiilor, cu spațiu între ele. Aveți mai jos trei exemple de rulare:

```
$ mpirun -np 3 ./testapd 10 3
2 5

$ mpirun -np 2 ./testapd 25 4
5 55

$ mpirun -np 8 ./testapd 3 0
1 2
```

În primul exemplu de mai sus, vectorul este [3,4,0,1,2,3,4,0,1,2], deci elementul căutat (adică 3) se află la pozițiile 0 și 5. În acest caz, numărul de apariții este 2, iar suma acestora este 5.

Atenție! Programul submis nu trebuie să printeze altceva decât numărul de apariții și suma pozițiilor aparițiilor, în această ordine și în formatul de mai sus. Afișarea se va face din orice task în afară de task-ul 0.

Atenție! Formula cu care se inițializează vectorul este doar pentru a ușura testarea. Nu o folosiți pentru a deduce matematic numărul de apariții. Fiecare task trebuie să itereze prin fragmentul de vector alocat lui și să găsească aparițiile valorii căutate.

Atenție! Doar task-ul 0 deține inițial vectorul. Celelalte task-uri nu cunosc nimic despre acesta inițial.

Atenție! Nerespectarea oricărei restricții de la punctele (de Atenție!) anterioare va face ca soluția să fie notată cu 0 (zero).

Se presupune că parametrii de intrare sunt corecți (atât ca număr și ordine, cât și ca valori). Valorile acceptate pentru N, P și X sunt: $0 < N <= 10000$, $0 < P <= 8$, $0 <= X < 5$. Programul trebuie să funcționeze corect pentru orice combinație de N, P și X din aceste intervale.

Trebuie să încărcați pe Moodle o arhivă care să conțină **în rădăcină** fișierele sursă, un Makefile cu target-ul *build* și eventual un fișier README.

Atenție! Fișierul README este necesar doar în cazul în care considerați că este nevoie de explicații adiționale în legătură cu implementarea realizată.

29 - Test practic APD

Să se implementeze, **folosind C/C++ și MPI**, un algoritm distribuit pentru determinarea numărului de valori prime dintr-un vector cu N elemente, precum și a sumei pozițiilor la care numerele prime se găsesc în vector, folosind P task-uri. Rezolvarea se va face maximizând concurența task-urilor. Rezultatul se va printa de către un task ales de către programator (poate fi thread-ul cu un anumit ID), dar **NU** de către task-ul 0.

Atenție! Task-ul 0 nu are voie să afișeze nimic. Nerespectarea acestei cerințe va duce la un punctaj de **0** pe problemă.

În urma compilării, va rezulta un executabil numit *testapd*, care se va rula astfel: **`mpirun -np P ./testapd N n1 n2 ... nN`** (n1…nN sunt cele N numere din vector). Programul va trebui să afișeze numărul de elemente prime din vector, apoi suma pozițiilor acestora, cu spațiu între ele. Aveți mai jos trei exemple de rulare:

```
$ mpirun -np 3 ./testapd 10 2 6 3 13 21 11 14 1 17 8
5 18

$ mpirun -np 4 ./testapd 3 13 14 15
1 0

$ mpirun -np 2 ./testapd 5 29 32 11 4 18
2 2
```

În primul exemplu de mai sus, vectorul este [2,6,3,13,21,11,14,1,17,8], deci elementele prime se află la pozițiile 0, 2, 3, 5 și 8. În acest caz, numărul de elemente prime este 5, iar suma pozițiilor acestora este 18.

Atenție! Programul submis nu trebuie să printeze altceva decât numărul de elemente prime și suma pozițiilor acestora, în această ordine și în formatul de mai sus. Afișarea se va face din orice task în afară de task-ul 0.

Atenție! Doar task-ul 0 are voie sa citească parametrii programului, deci doar el va cunoaște inițial elementele din vector și numărul lor.

Atenție! Nerespectarea oricărei restricții de la punctele (de Atenție!) anterioare va face ca soluția să fie notată cu 0 (zero).

Se presupune că parametrii de intrare sunt corecți (atât ca număr și ordine, cât și ca valori). Valorile acceptate pentru *N*, *P* și *ni* sunt: $0 < N <= 100$, $0 < P <= 8$, $0 <= ni < 100$. Programul trebuie să funcționeze corect pentru orice combinație de *N*, *P* și *ni* din aceste intervale.

Trebuie încărcată pe Moodle o arhivă care să conțină **în rădăcină** sursele, un Makefile cu target-ul *build* și eventual un fișier README.

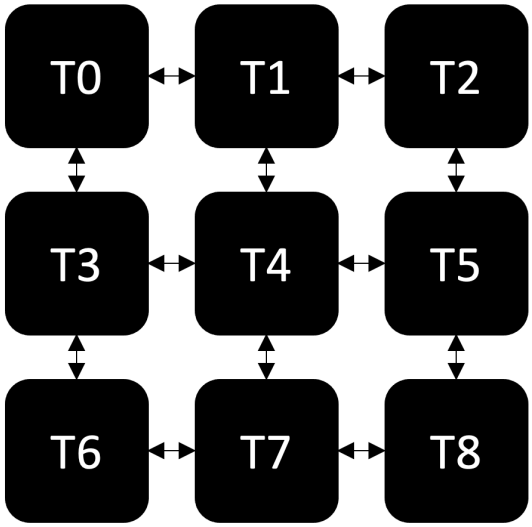
Atenție! Fișierul README este necesar doar în cazul în care considerați că este nevoie de explicații adiționale în legătură cu implementarea realizată. Indiferent dacă aveți sau nu fișier README, este important să adăugați comentarii în cod.

NEFOLOSITE

30 - Test practic APD

Să se implementeze, **folosind C/C++ și MPI** și presupunând **o topologie de tip tablou bi-dimensional cu P = N x N task-uri**, un program care să dubleze elementele dintr-o matrice de dimensiune kN x kN în mod echilibrat (adică fiecare task va dubla k^2 elemente). La final, se cere afișarea matricei dublate de către **task-ul cu rangul 0**. Comunicarea se poate realiza doar între task-uri vecine conform topologiei.

Puteți observa în imaginea de mai jos un exemplu de topologie pentru 3 x 3 task-uri, unde, de exemplu, task-ul 0 poate să comunice direct doar cu task-urile 1 și 3, task-ul 1 poate sa comunice direct doar cu task-urile 0, 2 și 4, etc.



În urma compilării, va rezulta un executabil numit *testapd*, care se va putea rula prin intermediul comenzii *mpirun -np P ./testapd k*. Astfel, programul va primi ca unic argument valoarea lui k, deoarece N poate fi dedus din P (care va fi întotdeauna un pătrat perfect). Este sarcina task-ului 0 să genereze matricea inițială de dimensiune kN x kN ca fiind `M[i][j] = i + j`, și apoi să o dea mai departe vecinilor săi și așa mai departe, până când fiecare task are informațiile necesare pentru calculele pe care trebuie să le facă. La final, task-ul 0 trebuie să afișeze matricea dublată. Aveți mai jos două exemple de rulare, unde output-ul este afișat de către task-ul cu rangul 0. Afișarea trebuie realizată pe formatul prezentat.

```
$ mpirun -np 9 ./testapd 2
0 2 4 6 7 10
2 4 6 8 10 12
4 6 8 10 12 14
6 8 10 12 14 16
8 10 12 14 16 18
10 12 14 16 18 20

$ mpirun -np 4 ./testapd 1
0 2
2 4
```

În primul exemplu de mai sus, P este 9, deci topologia arată ca cea din figura de mai sus (mai concret, ca o matrice de dimensiune 3 x 3). Valoarea lui k este 2, deci matricea care trebuie dublată este de dimensiune 6 x 6. Fiecare task trebuie deci să dubleze câte 4 elemente din matrice.

Atenție! Doar task-ul cu rangul 0 are voie să citească argumentele programului din *argv*, să genereze matricea inițială și să afișeze matricea dublată.

Atenție! Comunicația se va face strict cu task-urile vecine, conform topologiei.

Atenție! Nerespectarea oricărei restricții de la punctele (de Atenție!) anterioare va face ca soluția să fie notată cu 0 (zero).

Programul trebuie să funcționeze corect pentru orice P care este pătrat perfect și respectă 4 <= P <= 25, și pentru orice k ce respectă 1 <= k <= 100. Se poate presupune că argumentele date programului sunt corecte.

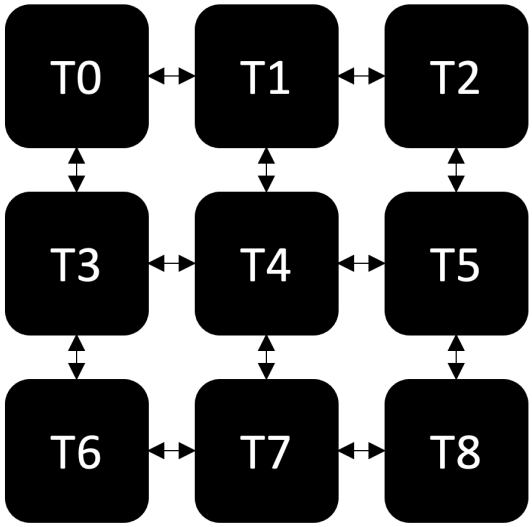
Trebuie să încărcați pe Moodle o arhivă care să conțină **în rădăcină** fișierele sursă, un Makefile cu target-ul *build* și eventual un fișier README.

Atenție! Fișierul README este necesar doar în cazul în care considerați că este nevoie de explicații adiționale în legătură cu implementarea realizată. Indiferent dacă aveți sau nu fișier README, este important să adăugați comentarii în cod.

31 - Test practic APD

Să se implementeze, **folosind C/C++ și MPI** și presupunând **o topologie de tip tablou bi-dimensional cu P = N x N task-uri**, un program care să numere aparițiile unui element într-un vectorul de dimensiune kP într-un mod cât mai echilibrat. La final, se cere afișarea numărului de apariții de către **task-ul cu rangul 0**. Comunicarea se poate realiza doar între task-uri vecine conform topologiei.

Puteți observa în imaginea de mai jos un exemplu de topologie pentru 3 x 3 task-uri, unde, de exemplu, task-ul 0 poate să comunice direct doar cu task-urile 1 și 3, task-ul 1 poate sa comunice direct doar cu task-urile 0, 2 și 4, etc.



În urma compilării, va rezulta un executabil numit *testapd*, care se va putea rula prin intermediul comenzii *mpirun -np P ./testapd E k*. Astfel, programul va primi ca argumente numărul căutat E și valoarea lui k. Este sarcina task-ului 0 să genereze vectorul de dimensiune kP ca **`v[i] = (i + 3) % 7`** și apoi să se asigure că fiecare task are informațiile necesare pentru o căutare echilibrată. La final, task-ul 0 trebuie să afișeze numărul total de apariții ale valorii căutate în vector. Aveți mai jos două exemple de rulare, unde output-ul este afișat de către task-ul cu rangul 0. Afișarea trebuie realizată pe formatul prezentat.

```
$ mpirun -np 9 ./testapd 2 2
2

$ mpirun -np 16 ./testapd 5 3
7
```

În primul exemplu de mai sus, P este 9, deci topologia arată ca cea din figura de mai sus (mai concret, ca o matrice de dimensiune 3 x 3). Valoarea lui k este 2, deci vectorul în care se caută valoarea 2 va avea 18 elemente.

Atenție! Doar task-ul cu rangul 0 are voie să citească argumentele programului din *argv*, să genereze vectorul și să afișeze rezultatul final.

Atenție! Comunicația se va face strict cu task-urile vecine, conform topologiei.

Atenție! **Nerespectarea oricărei restricții de la punctele (de Atenție!) anterioare va face ca soluția să fie notată cu 0 (zero).**

Programul trebuie să funcționeze corect pentru orice P care este pătrat perfect și respectă 4 <= P <= 25, și pentru orice k ce respectă 1 <= k <= 100. Se poate presupune că argumentele date programului sunt corecte.

Trebuie să încărcați pe Moodle o arhivă care să conțină **în rădăcină** fișierele sursă, un Makefile cu target-ul *build* și eventual un fișier README.

Atenție! Fișierul README este necesar doar în cazul în care considerați că este nevoie de explicații adiționale în legătură cu implementarea realizată. Indiferent dacă aveți sau nu fișier README, este important să adăugați comentarii în cod.